

Measuring the Measurement: Repeated-Run Reliability of LLM Code Evaluation

Anonymous authors

Paper under double-blind review

Abstract

1 Evaluation scores are measurements, and like any measurement they have
 2 a reliability that is rarely quantified. We ask whether single-run pass
 3 rate—the dominant instrument for large language model (LLM) code
 4 evaluation—is a reliable measurement of model capability, or whether
 5 it conceals substantial test–retest variation. Treating each problem as a
 6 repeated measurement, we introduce a repeated-run protocol with three
 7 instruments: run-level pass rate (RLPR), perfect stability rate (PSR, the
 8 fraction of tasks solved on *every* retry), and average per-problem variance
 9 (AV). On a recency-controlled benchmark of 100 LeetCode-style problems,
 10 we evaluate 16 models from five provider families under two prompt tem-
 11 plates with five repeated runs each (16,000 evaluation instances). Although
 12 RLPR and PSR are strongly correlated ($r = 0.985$), run-level pass rate over-
 13 states retry-free coverage by up to 17.8 percentage points, the gap is largest
 14 for mid-performing systems, and it reverses model rankings even among
 15 closely matched systems. Prompt effects are model-dependent rather than
 16 uniformly beneficial. We argue that reporting the test–retest reliability of
 17 a score is a necessary part of valid LLM evaluation, with deterministic
 18 single-turn tasks as the tractable i.i.d. baseline for harder non-stationary
 19 settings.

20 1 Introduction

21 An evaluation score is a measurement, and every measurement instrument has a *reliability*:
 22 the degree to which it returns the same value when the same quantity is measured again.
 23 For large language models (LLMs), the dominant instrument—single-run pass rate on a
 24 held-out benchmark—is almost always reported as a point estimate, as if it were perfectly
 25 repeatable. Yet LLM decoding is stochastic, so re-running the same model on the same
 26 task is a *repeated measurement* that need not agree with itself. Borrowing the language
 27 of psychometrics and metrology, we ask a measurement-science question: what is the
 28 *test–retest reliability* of pass-rate evaluation, and how much does a single run mislead us
 29 about a model’s underlying capability? We study this in the cleanest available setting—
 30 deterministic, single-turn programming tasks graded by a fixed verifier—where the only
 31 source of disagreement between repeated measurements is the model itself.

32 Large language models are increasingly used for text-conditioned tasks with deterministic
 33 specifications: given a fixed textual input and a fixed verifier, correctness is binary. Program
 34 synthesis and code assistance are especially important instances of this broader setting,
 35 because they pair natural-language problem statements with executable test suites. Conven-
 36 tional benchmarks typically report single-run performance, or summarize repeated sam-
 37 pling through eventual-success metrics such as $\text{pass}@k$ (Chen et al., 2021; Austin et al., 2021).
 38 Recent work has shown that repeated sampling can be scaled as a form of inference-time
 39 compute (Brown et al., 2025), yet the *stability* of outcomes across such repeated invocations—
 40 whether a model consistently succeeds or fails on the same task—has received less systematic
 41 attention. In practice, repeated calls under the same task description can yield different
 42 outputs and different pass/fail outcomes, even when the downstream workflow expects
 43 reproducible behavior (Atil et al., 2024; Alvarado Gonzalez et al., 2025).

This paper studies that repeated-invocation variability directly. Rather than treating repeated sampling only as an opportunity to recover one success among many attempts, we investigate the relationship between correctness and stability under fixed conditions. Our focus is therefore complementary to $\text{pass}@k$ (Chen et al., 2021): whereas $\text{pass}@k$ asks whether a model can produce *at least one* correct solution in k attempts—a capability measure—we ask how reliably it succeeds on *every* invocation, a deployment reliability measure. These two questions target different downstream concerns and can yield divergent rankings, particularly for mid-accuracy systems.

The paper is organized around three measurement-validity questions:

1. How large is the gap between a model’s run-level pass rate and the fraction of tasks it solves reliably on *every* retry—that is, how much measurement error does single-run reporting hide?
2. Which aggregate summaries best expose the per-task measurement error (test–retest variability) that a single pass-rate number conceals?
3. As an exploratory analysis: do design choices—prompt template, reasoning-specialized architectures—systematically change the reliability of the measurement?

We focus on deterministic tasks because they represent a broader and practically important class of NLP system behavior: the model is conditioned on text, but the output is consumed by a verifier that expects reproducible correctness. Code-generation systems are a particularly stringent instance of this pattern, alongside tasks such as text-to-SQL (Gao et al., 2024), structured semantic parsing, tool-call argument generation, and formal text transformations. In such settings, the accuracy–stability gap has direct practical consequences: a model with 60% run-level pass rate but only 47% perfect stability solves many individual attempts, yet still leaves a substantial fraction of tasks in a retry-required regime. Our aim is therefore not merely to rank models by mean performance, but to characterize how much of their observed accuracy translates into reliable repeated-use behavior. We deliberately restrict attention to deterministic, single-turn tasks because they are the tractable i.i.d. case of repeated measurement: each run is an independent draw under a fixed verifier. This is the natural baseline for the harder, non-stationary regimes—interactive, path-dependent, or adaptively optimized evaluation—where measurement reliability is even less understood, and where the analysis tools developed here would need to be extended.

2 Related Work

Code-generation evaluation is commonly framed around functional correctness and $\text{pass}@k$ under automated testing (Chen et al., 2021; Austin et al., 2021). Foundational benchmarks emphasize whether a model eventually produces at least one correct program under repeated sampling (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021), while more recent efforts address contamination and temporal validity (Jain et al., 2025), with parallel work extending evaluation to real-world repository-level issues (Jimenez et al., 2024). Prompting and inference-time interventions use additional computation to improve performance rather than to measure it: zero-shot chain-of-thought (Kojima et al., 2022) restructures the prompt, while self-consistency and adaptive-consistency (Wang et al., 2023; Aggarwal et al., 2023) aggregate multiple samples. In parallel, work on LLM output non-determinism shows that performance varies substantially across repeated runs under identical conditions (Atil et al., 2024; Alvarado Gonzalez et al., 2025), and prompt sensitivity research demonstrates that small phrasing changes can shift accuracy (Zhuo et al., 2024).

Perfect Stability Rate (PSR) is mathematically equivalent to $\text{pass}@k$ at $k = R$ (the number of repeated runs) with an all-successes threshold; the distinction is interpretive. $\text{Pass}@k$ estimates $P(\text{at least one success in } k \text{ draws})$ —a *capability* measure—while PSR estimates $P(\text{all } k \text{ draws succeed})$ —a *deployment reliability* measure. These quantities diverge most sharply for mid-accuracy systems. The contribution lies in treating PSR and Average Variance (AV) as first-class evaluation targets for deterministic code tasks—both defined formally in Section 3.5—and using them to characterize the accuracy–stability gap directly.

Recent work emphasizes that code evaluation is sensitive to benchmark construction and reporting choices (Liu et al., 2023; Du et al., 2024). Our framing shifts the emphasis from the *construction* validity of a benchmark to the *reliability* of the measurement it produces: we treat an evaluation as an instrument whose test–retest behavior can be quantified, in the spirit of reliability theory in psychometrics and metrology, rather than reported as a single repeatable number. Work on confidence calibration (Guo et al., 2017) and performance–efficiency trade-offs (Zhang et al., 2026) reflects a broader recognition that average accuracy alone is an incomplete measure of model reliability. This perspective is deliberately narrower than end-to-end agent evaluation: it isolates repeated-run behavior for single-turn coding, while broader orchestration, tool use, and runtime dependencies require separate analysis.

3 Experimental Setup

3.1 Problem Dataset

We evaluate on a curated dataset of deterministic programming problems, denoted $\mathcal{D} = \{(x_i, T_i)\}_{i=1}^N$, where x_i is the natural-language specification (including I/O format and constraints) and T_i is a deterministic test harness. Each problem is graded by executing the model-produced program against T_i , producing a binary correctness indicator. These tasks are instantiated as LeetCode problems with platform-provided acceptance criteria, and all generations are evaluated as Python solutions against Python-based submission templates.

Problem selection. To reduce the risk of benchmark contamination from model training data, we select the $N = 100$ most recently published problems available on the platform at collection time (newest first), excluding paid-only problems; no further curation is applied, so the benchmark is determined entirely by recency rank. This substantially reduces the probability of direct memorization relative to historically popular problems, but does not guarantee a contamination-free evaluation—recently published problems are often discussed online within days of release, and we do not perform contamination-detection analysis (Deng et al., 2024). We return to this in the Limitations section.

The dataset spans three difficulty tiers: 20 Easy, 50 Medium, and 30 Hard problems, covering a broad range of algorithmic topics including Array, Dynamic Programming, String, Graph, and Simulation. All problems satisfy the following properties:

- **Deterministic grading:** each T_i yields a deterministic accept/reject outcome.
- **Self-contained evaluation:** no network access or external dependencies are required at runtime.
- **Fixed snapshot:** all models are evaluated on the identical problem set to ensure comparability across runs.
- **No personal data:** problems consist of algorithmic specifications only and contain no personally identifying information.

In the evaluation framework, this benchmark is stored as a fixed local dataset snapshot rather than fetched online at run time. This design ensures that all model families are evaluated against the same problem set and ordering, and reduces the chance that later platform changes silently alter the benchmark during the experimental campaign.

3.2 Models

We evaluate 16 models across five provider families (OpenAI, Anthropic, Google, Qwen, DeepSeek), covering a range of model scales, training objectives, and architectural choices, including both standard and reasoning-specialized variants. Four are reasoning-specialized (o4-mini, Gemini 2.5 Flash+Think, QwQ-Plus, DeepSeek-R1). The full roster with types is given in Table 2, and exact API identifiers and access dates in Appendix C.

3.3 Evaluation Framework

For a model m and a fixed evaluation configuration c (prompt template, decoding parameters, toolchain), we run each problem x_i multiple times. Each run produces a candidate program which is executed against T_i .

Let $Y_{i,r}^{(m,c)} \in \{0, 1\}$ denote whether run $r \in \{1, \dots, R\}$ passes all tests for problem i . We treat compilation errors, runtime errors, timeouts, and wrong answers as failures (0). The pipeline separates generation from execution—outputs are generated via provider APIs, cached, normalized to a shared format, and only then submitted to the deterministic judge—which makes runs resumable and avoids conflating transient provider failures with execution outcomes.

3.4 Repeated-Run Methodology

We adopt a repeated-run design to estimate both mean performance and variability. For each (m, c) pair we:

1. Sample R independent generations per problem under configuration c .
2. Evaluate each generation deterministically against T_i .
3. Aggregate outcomes into run-level and problem-level metrics with uncertainty estimates.

The primary controlled intervention is the prompt template. We compare two prompt variants while holding the dataset, model, decoding parameters, and execution environment fixed. All models are evaluated with temperature $T = 0.3$, top- $p = 0.9$, and $R = 5$ repeated runs per problem per prompt configuration. The default maximum output budget is 4,096 tokens; reasoning-specialized models (o4-mini, Gemini 2.5 Flash+Think) use larger budgets to accommodate extended chain-of-thought output (see Appendix C for per-model details). The o-series API (o4-mini) does not support explicit temperature or top- p settings; decoding parameters are omitted for that model (implications discussed in the Limitations section). We use a non-zero temperature because the goal of the study is to characterize repeated-run variability under realistic stochastic decoding rather than to collapse outputs toward near-deterministic behavior at $T = 0$. The two prompt templates—DETAILED and MINIMAL—are held fixed across all models to enable a controlled comparison of prompt sensitivity.

Prompt templates. Both prompts instruct the model to return plain Python source code only and to follow the provided LeetCode method signature exactly. The DETAILED prompt uses a longer structured template emphasizing careful constraint reading, edge-case coverage, submission readiness, and accepted-style performance. The MINIMAL prompt conveys the same task more compactly, with lighter instruction scaffolding. Because both templates share the same output constraints and code template, the prompt comparison is a controlled low-contrast comparison that isolates instruction density rather than changing the task itself. Accordingly, the study does not test broad prompt-engineering strategies; it evaluates a narrow prompt variation under otherwise fixed conditions.

3.5 Metrics Definition

Run-Level Pass Rate (RLPR).

$$\text{RLPR}(m, c) = \frac{1}{NR} \sum_{i=1}^N \sum_{r=1}^R Y_{i,r}^{(m,c)}.$$

RLPR measures the probability that a *random invocation* succeeds.

Perfect Stability Rate (PSR).

$$\text{PSR}(m, c) = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[\sum_{r=1}^R Y_{i,r}^{(m,c)} = R \right].$$

Table 1: Main results across all 16 models and two prompt configurations (100 problems, $R = 5$ runs, 16,000 total evaluation instances). RLPR and PSR are in %; AV is shown $\times 10^{-2}$ (i.e., multiply by 0.01 for the raw value). All entries are point estimates; 95% CIs (Wilson score for RLPR/PSR; bootstrap for AV) available from authors on request. † reasoning-specialized model. * output budget differs from the 4,096-token default: Gemini 2.5 Flash+Think used 24,576 tokens (8,192 thinking); o4-mini used 16,384 tokens. ‡ o4-mini decoding is not user-controllable; its PSR/AV may reflect reduced stochasticity rather than task reliability—see the Limitations section.

Family	Model	RLPR (%)		PSR (%)		AV ($\times 10^{-2}$)	
		D	M	D	M	D	M
OpenAI	GPT-4.1	59.6	59.2	47.5	49.0	4.36	4.08
	GPT-4.1-mini	58.4	59.2	50.0	47.0	3.28	4.48
	o4-mini ^{†‡}	86.2	85.6	75.0	69.0	3.36	5.20
Anthropic	Claude Sonnet 4.5	52.8	51.4	43.0	47.0	3.36	1.68
	Claude Haiku 4.5	41.2	40.8	35.0	34.0	2.40	2.00
Google	Gemini 3.1 Pro (preview)	86.2	85.8	75.0	74.0	3.36	3.92
	Gemini 3 Flash (preview)	80.8	83.2	67.0	70.0	5.28	4.40
	Gemini 3.1 Flash-Lite (preview)	69.2	63.6	53.0	50.0	5.28	5.44
	Gemini 2.5 Flash*	53.8	48.2	36.0	33.0	7.04	5.84
	Gemini 2.5 Flash+Think ^{†*}	71.6	74.4	58.0	57.0	4.64	4.72
Qwen	QwQ-Plus	72.8	72.6	59.0	58.0	4.80	4.72
	Qwen-Plus	63.4	61.2	52.0	51.0	4.72	4.64
	Qwen-Max	30.6	30.4	26.0	25.0	2.24	2.24
	Qwen-Turbo	34.4	30.8	32.0	28.0	0.88	1.04
DeepSeek	DeepSeek-R1	84.2	84.4	74.0	75.0	3.76	3.20
	DeepSeek-V3.2	48.4	47.6	36.0	38.0	5.52	4.00

182 PSR quantifies the fraction of tasks for which the system is reliably correct without retries.

183 **Average Variance (AV).** $AV(m, c) = \frac{1}{N} \sum_{i=1}^N \frac{R}{R-1} \hat{p}_i (1 - \hat{p}_i)$ with $\hat{p}_i = \frac{1}{R} \sum_r Y_{i,r}$ captures
 184 average instability across tasks.

185 **Inference.** We report 95% Wilson score intervals for RLPR and PSR, and nonparametric
 186 bootstrap CIs for AV. Since both prompt templates are evaluated on the same problems, we
 187 compare PSR across prompt conditions using exact McNemar tests on the binary indicator of
 188 whether all $R = 5$ runs pass. Reported statistics are descriptive summaries of the observed
 189 protocol rather than universal claims; prompt comparisons are specific to the two templates
 190 used here.

191 4 Results

192 4.1 Overall Accuracy and Stability

193 Table 1 reports RLPR, PSR, and AV for all 32 model/prompt configurations. The top
 194 performers under RLPR are o4-mini (86.2% D / 85.6% M), Gemini 3.1 Pro (86.2% D / 85.8%
 195 M), and DeepSeek-R1 (84.2% D / 84.4% M). More broadly, the table shows that high run-
 196 level performance and high retry-free coverage often co-occur, but not in a one-to-one way:
 197 models with similar RLPR can still differ materially in PSR and AV.

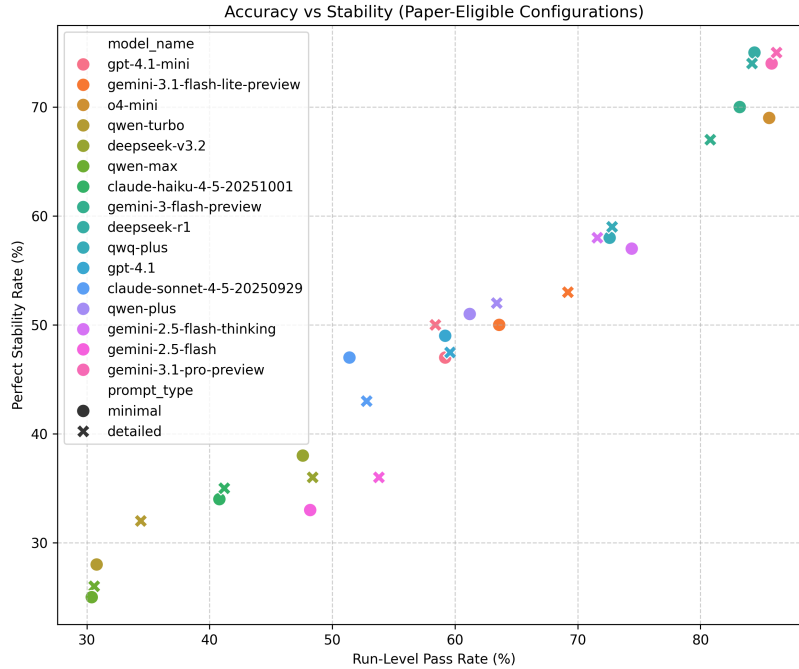


Figure 1: RLPR vs. PSR scatter plot across all 32 configurations (16 models $\times$ 2 prompt types). Points are colored by model family and shaped by prompt type. The diagonal marks PSR = RLPR; points below indicate configurations where mean success overstates stable problem coverage.

Figure 1 plots RLPR against PSR for all 32 configurations. All points fall below the diagonal, confirming that RLPR consistently overstates production reliability. Across all configurations, RLPR and PSR are strongly correlated ($r = 0.985$, $p < 0.001$), which is directionally expected because both metrics are derived from the same underlying per-problem success structure. The informative result is therefore not the existence of a high correlation itself, but the persistent and non-uniform gap between run-level success and retry-free coverage.

The gap $\Delta = \text{RLPR} - \text{PSR}$ is non-uniform and reaches its maximum in the mid-accuracy tier. Gemini 2.5 Flash has the largest gap (17.8 pp under DETAILED), followed by Gemini 3.1 Flash-Lite (16.2 pp D) and Gemini 2.5 Flash+Think (17.4 pp M). At the low end, Qwen-Turbo has a gap of only 2.4 pp—not because it is uniformly reliable, but because its outcomes are highly polarized: many problems are either always failed or always passed, leaving less room for intermediate trial-to-trial variability. This pattern is reflected in AV: Qwen-Turbo has the lowest AV (0.0088), while Gemini 2.5 Flash has the highest (0.0704). Together, PSR and AV help distinguish between systems that are consistently strong, consistently weak, and unstable in the middle.

While $\text{RLPR} \geq \text{PSR}$ is mathematically expected for any non-trivial Bernoulli process, and Bernoulli variance peaks at $p \approx 0.5$ so mid-accuracy models exhibit the largest gaps by construction, the *magnitude* of Δ and *which* models fall in the mid-accuracy danger zone cannot be derived a priori, since per-problem success probabilities are unknown in advance. The empirical contribution is calibrating this expected pattern at scale: the gap is observable, model-specific, and meaningfully large (up to 17.8pp), so RLPR rankings alone cannot reveal how much of a model’s accuracy translates into retry-free reliability.

220 4.2 Problem-Level Stability Analysis

221 Aggregate rates hide qualitatively different reliability profiles. A cross-model stability
 222 heatmap (Figure 2) and the per-problem pass-rate distribution (Figure 3), both in Ap-
 223 pendix A, show that models with similar mean pass rates can differ sharply in how their
 224 successes are distributed: a model with many $\hat{p}_i \approx 1$ and $\hat{p}_i \approx 0$ problems behaves very
 225 differently from one with many $\hat{p}_i \approx 0.4$ – 0.6 problems. Qwen-Turbo shows the most polar-
 226 ized profile ($AV = 0.0088$), while Gemini 2.5 Flash has the highest AV (0.0704). Retry-based
 227 recovery does not eliminate this instability: PSR measures the fraction of tasks needing no
 228 retries, while AV quantifies how much of the remaining workload sits in a stochastic regime.

229 4.3 Difficulty Gradient and Failure Modes

230 Pooling outcomes by benchmark difficulty reveals a pronounced gradient: run-level pass
 231 rates are highest on Easy problems (90.6%), lower on Medium (62.2%), and substantially
 232 lower on Hard (34.0%). The dominant non-accept verdict is *Wrong Answer* ($\sim 58\%$), fol-
 233 lowed by *Time Limit Exceeded* ($\sim 27\%$) and *Runtime Error* ($\sim 11\%$), with the remaining $\sim 4\%$
 234 comprising compile errors and other verdicts. Most failures thus arise from incorrect al-
 235 gorithms rather than formatting noise, though timeouts and runtime failures increase on
 236 harder problems, indicating that instability is also tied to implementation robustness. A
 237 low PSR can therefore reflect two regimes—algorithmic inconsistency and implementation
 238 fragility—which imply different mitigation strategies.

239 4.4 Reasoning vs. Standard Models (Exploratory)

240 We tested whether reasoning-specialized models (DeepSeek-R1, QwQ-Plus, o4-mini, Gemini
 241 2.5 Flash+Think) exhibit smaller accuracy–stability gaps after controlling for accuracy, by
 242 regressing Δ on RLPR and comparing group residuals. They do not: the mean residual
 243 for the eight reasoning configurations (-0.04) is statistically indistinguishable from the
 244 24 standard configurations ($+0.01$; permutation test $p = 0.97$), and mean AV is likewise
 245 comparable (4.30 vs. 3.81×10^{-2}). The pattern is model-specific—DeepSeek-R1 is markedly
 246 more stable than expected for its accuracy while Gemini 2.5 Flash+Think is less so—and
 247 severely underpowered at four models, so we read $p = 0.97$ as “insufficient evidence,” not
 248 “no effect.” Full analysis and Figure 4 appear in Appendix A.

249 4.5 Prompt Sensitivity (Exploratory)

250 Comparing DETAILED vs. MINIMAL prompting, effects are model-dependent and small: 11
 251 of 16 models favor DETAILED and 5 favor MINIMAL, with a mean absolute PSR difference
 252 of ~ 2.3 pp and no model reaching $p < 0.05$ under exact McNemar tests (Zhuo et al., 2024).
 253 Because both templates share identical structure and differ only in instruction density, this is
 254 a deliberately low-contrast comparison and underpowered at $N = 100$; meaningful effects
 255 cannot be ruled out. Full analysis and Figure 5 appear in Appendix A.

256 4.6 Accuracy–Stability Gap

257 Beyond its magnitude, the gap Δ can reorder model rankings. Because Δ peaks in the
 258 mid-accuracy tier (as shown earlier), two models with similar RLPR can differ substantially
 259 in retry-free reliability, so a single RLPR ranking cannot infer the accuracy–stability gap;
 260 PSR makes the distinction explicit.

261 RLPR and PSR can disagree on model rankings despite the high correlation. For example,
 262 GPT-4.1 leads GPT-4.1-mini on RLPR (59.6% vs. 58.4%) but trails on PSR (47.5% vs. 50.0%)—a
 263 reversal driven by GPT-4.1’s larger accuracy–stability gap (12.1 pp vs. 8.4 pp). This confirms
 264 that PSR captures deployment-relevant distinctions invisible to RLPR rankings alone.

265 5 Conclusion

266 We presented a repeated-run evaluation framework for deterministic programming tasks
 267 and used it to study the relationship between correctness and stability. Across 16 models, 100
 268 problems, and 5 repeated runs per configuration (16,000 total instances), run-level pass rate
 269 and retry-free coverage are strongly correlated, but run-level accuracy consistently exceeds
 270 perfect stability. The accuracy–stability gap is non-uniform and reaches up to 17.8 percentage
 271 points, peaking in the mid-accuracy tier. Importantly, PSR and RLPR can reverse model
 272 rankings even at high correlation ($r = 0.985$): GPT-4.1 leads GPT-4.1-mini on RLPR but
 273 trails on PSR, demonstrating that the two capture deployment-relevant distinctions invisible
 274 to single-run reporting. We view these metrics as complementary: RLPR characterizes
 275 average invocation success, while PSR and AV reveal whether that success is concentrated
 276 in consistently solved problems or dispersed across unstable outcomes. Prompt effects
 277 and the reasoning-vs-standard distinction are both heterogeneous and exploratory at the
 278 current sample size, suggesting that neither is a reliable lever for improving stability without
 279 model-specific tuning.

280 Future work should replicate the repeated-run analysis on non-coding deterministic NLP
 281 tasks (e.g., text-to-SQL) and control output token budgets to cleanly isolate the effect of
 282 reasoning on stability; broader orchestration and runtime dependencies require separate
 283 analysis (Jiang et al., 2026). Taken together, these findings suggest that reporting the test-
 284 retest reliability of a score—not just its point value—is a necessary part of valid evaluation
 285 for any deterministic text-conditioned generation task, and a prerequisite for extending
 286 measurement to the harder non-stationary settings.

287 Limitations

288 Several limitations qualify our results. First, the benchmark is small ($N = 100$) and restricted
 289 to recent LeetCode-style problems: this reduces overlap with heavily circulated historical
 290 benchmarks but does not eliminate contamination risk, since recent contest problems are
 291 often discussed online within days of release. The size is also a tradeoff—16 models $\times$ 2
 292 prompts $\times$ 5 runs still yields 16,000 judged instances, ample for broad correctness–stability
 293 patterns but underpowered for small prompt effects (consistent with the non-significant
 294 comparisons in Section 4.5). Second, the protocol is intentionally narrow: we vary prompt
 295 template and repeated sampling while holding the harness fixed, so it does not characterize
 296 broader prompt engineering, tool use, self-repair, retrieval, test-time selection, or multi-turn
 297 interaction. Third, access is mediated through provider APIs, so despite deterministic
 298 grading we cannot control backend changes or model refreshes; conclusions should be read
 299 relative to the documented snapshots and access dates, not as timeless statements about a
 300 provider. Fourth, our metrics compress distinct failure modes into a binary outcome and
 301 do not by themselves explain whether instability arises from reasoning errors, formatting
 302 mistakes, runtime failures, or judge edge cases. Fifth, a caveat applies to o4-mini: the
 303 o-series API exposes no temperature/top- p controls, so if its effective decoding is near-
 304 deterministic, its PSR/AV measure the near-absence of stochasticity rather than stability
 305 under stochastic sampling, and should be interpreted cautiously. Finally, results are limited
 306 to Python code generation on deterministic tasks with executable tests, and should not be
 307 read as a complete account of reliability for other languages, open-ended generation, or
 308 production systems with additional orchestration layers.

309 Ethics Statement

310 This paper evaluates commercial and open-access LLM APIs on deterministic programming
 311 tasks. The study does not involve human subjects, personal data collection, or annotation
 312 labor. The main ethical considerations are instead tied to platform use and deployment
 313 interpretation. First, the evaluation relies on automated submission to an external online
 314 judge. Such use should be understood as research benchmarking under the platform’s
 315 operational constraints, and any public release should avoid exposing credentials, session
 316 artifacts, or procedures that would enable abusive automation. Second, the evaluated

models are accessed through third-party APIs whose outputs and availability may be governed by provider-specific terms of service and may change over time. Our reported results should therefore be interpreted as measurements of accessed systems under a documented evaluation protocol, not as immutable properties of any provider’s model family.

More broadly, stronger performance on coding benchmarks can increase both beneficial and harmful capabilities. Better repeated-run reliability can support legitimate software engineering workflows, but it may also lower the barrier to generating code in settings where security review is weak. For this reason, we frame our contribution as an evaluation methodology rather than as an endorsement of fully autonomous code deployment.

References

- Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let’s sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12375–12396, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.761.
- Miguel Angel Alvarado Gonzalez, Michelle Bruno Hernandez, Miguel Angel Peñaloza Perez, Bruno Lopez Orozco, Jesus Tadeo Cruz Soto, and Sandra Malagon. Do repetitions matter? Strengthening reliability in LLM evaluations, 2025.
- Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Non-determinism of “deterministic” LLM settings, 2024.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=0xUEBQV54B>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgén Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. Investigating data contamination in modern benchmarks for large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 8706–8719, Mexico City, Mexico, 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.naacl-long.482/>.
- Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024. doi: 10.1145/3597503.3639219. URL <https://dl.acm.org/doi/10.1145/3597503.3639219>.

- 369 Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou.
370 Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings*
371 *of the VLDB Endowment*, 17(5):1132–1145, 2024. doi: 10.14778/3641204.3641221.
- 372 Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern
373 neural networks. In *Proceedings of the 34th International Conference on Machine Learning*,
374 volume 70 of *Proceedings of Machine Learning Research*, pp. 1321–1330, 2017. URL <https://proceedings.mlr.press/v70/guo17a.html>.
375
- 376 Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo,
377 Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring
378 coding challenge competence with APPS. In *Proceedings of the Neural Information Processing*
379 *Systems Track on Datasets and Benchmarks*, 2021. URL <https://openreview.net/forum?id=sD93G0zH3i5>.
380
- 381 Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang,
382 Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and
383 contamination free evaluation of large language models for code. In *The Thirteenth*
384 *International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
385
- 386 Xiaochong Jiang, Shiqi Yang, Wenting Yang, Yichen Liu, and Cheng Ji. SOK: A taxonomy of
387 attack vectors and defense strategies for agentic supply chain runtime, 2026.
- 388 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and
389 Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub
390 issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
391 URL <https://openreview.net/forum?id=VTF8yNQm66>.
- 392 Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and
393 Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Ad-*
394 *vances in Neural Information Processing Systems*, volume 35, pp. 22199–22213,
395 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html.
396
- 397 Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code
398 generated by ChatGPT really correct? rigorous evaluation of large language mod-
399 els for code generation. In *Advances in Neural Information Processing Systems*, vol-
400 ume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html.
401
- 402 Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha
403 Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning
404 in language models. In *The Eleventh International Conference on Learning Representations*
405 *(ICLR)*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- 406 Yike Zhang, Zuodong Xiang, and Hailu Xu. Performance-efficiency trade-offs in human
407 preference prediction: A comparative study of traditional machine learning and large lan-
408 guage models. In *Proceedings of the 31st IEEE Symposium on Computers and Communications*
409 *(ISCC)*. IEEE, 2026. To appear.
- 410 Jingming Zhuo, Songyang Zhang, Xinyu Fang, Haodong Duan, Dahua Lin, and Kai Chen.
411 ProSA: Assessing and understanding the prompt sensitivity of LLMs. In *Findings of the*
412 *Association for Computational Linguistics: EMNLP 2024*, pp. 1950–1976, Miami, Florida,
413 USA, 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.findings-emnlp.108/>.
414

415 A Additional Stability Analyses

416 This appendix contains the detailed problem-level, reasoning-vs-standard, and prompt-
417 sensitivity analyses summarized in the Results section, together with their figures.

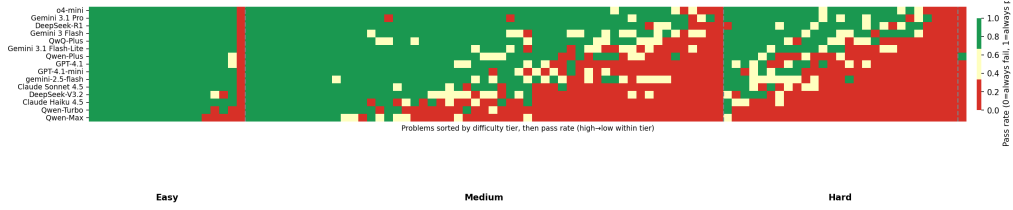


Figure 2: Cross-model stability heatmap (DETAILED prompt). Rows: 16 models sorted by RLPR descending. Columns: 100 problems grouped by difficulty tier with dashed separators, and ordered within each tier by decreasing mean pass rate. Cell color encodes empirical pass rate over $R = 5$ runs (green = 1.0, red = 0.0, yellow ≈ 0.5).

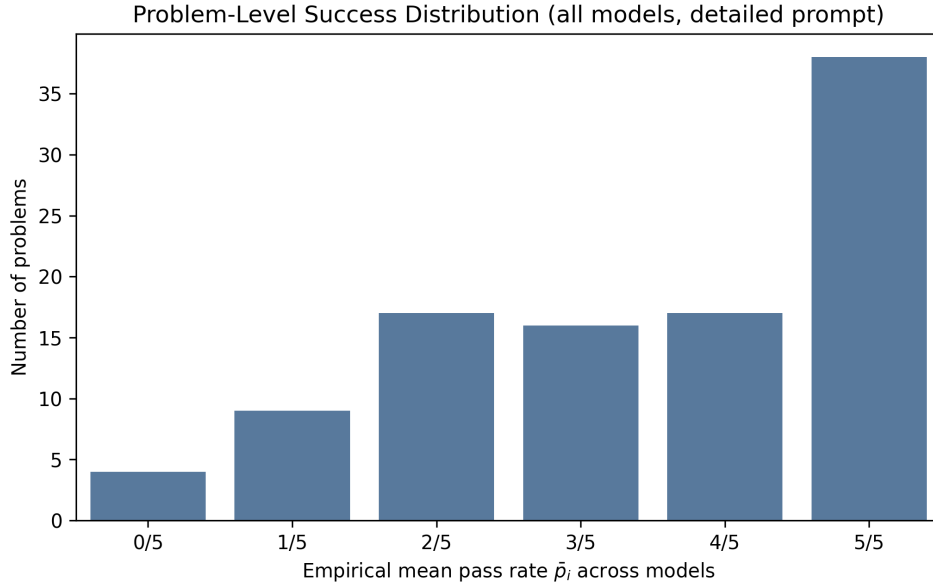


Figure 3: Problem-level success distribution (DETAILED prompt). The figure contains one panel per model (16 panels total), arranged in decreasing order of PSR. Each panel shows the fraction of that model’s 100 problems falling in each empirical pass-probability bin $\hat{p}_i \in \{0/5, 1/5, \dots, 5/5\}$, where 0/5 means the model failed all 5 runs and 5/5 means it passed all 5.

418 A.1 Problem-Level Stability Detail

419 Figure 2 presents a cross-model stability overview. Each row corresponds to one of the
 420 16 models, ordered by RLPR under DETAILED prompting (highest at top). Each column
 421 corresponds to one of the 100 problems, grouped by difficulty tier (Easy, Medium, Hard)
 422 and sorted within each tier by decreasing mean pass rate across all models. Cell color
 423 encodes the empirical pass probability for that model–problem pair, averaged over $R = 5$
 424 runs: green indicates a problem solved consistently on every run, red indicates consistent
 425 failure, and yellow indicates high trial-to-trial variability.

426 Figure 3 shows the distribution of per-problem pass rates aggregated across all models. This
 427 view exposes qualitatively different reliability profiles that similar aggregate pass rates can
 428 hide: a model with many $\hat{p}_i \approx 1$ and $\hat{p}_i \approx 0$ problems behaves differently from a model
 429 with many $\hat{p}_i \approx 0.4$ or 0.6 problems, even when their average run-level pass rate is similar.

A.2 Reasoning vs. Standard Models: Full Analysis

Reasoning-specialized models generate an internal chain-of-thought before producing a final answer, which could plausibly improve output consistency by structuring the solution process. We test whether this architectural distinction manifests as a systematically smaller accuracy–stability gap after controlling for overall accuracy level.

Grouping. We treat four models as reasoning-specialized (marked † in Table 2): DeepSeek-R1, QwQ-Plus, o4-mini, and Gemini 2.5 Flash+Think. The remaining 12 systems are classified as standard models. All conclusions from this analysis remain exploratory given the modest group sizes.

RLPR-controlled gap analysis. Because $\Delta = \text{RLPR} - \text{PSR}$ is itself correlated with RLPR (higher-accuracy models tend to have larger absolute gaps, as shown in Section 4.6), a raw comparison of mean gaps between groups would be confounded by accuracy level. To address this, we fit a linear model $\Delta \sim \text{RLPR}$ across all 32 configurations ($R^2 = 0.40$, $p < 0.001$) and examine the residuals: a negative residual indicates a smaller gap than expected at the model’s accuracy level, i.e., greater stability relative to accuracy.

Figure 4 shows the result. Reasoning models show strongly heterogeneous behavior: DeepSeek-R1 falls well below the regression line (mean residual ≈ -4.4), suggesting markedly better stability than expected for its accuracy level; o4-mini is near the line (residual ≈ -0.5); while QwQ-Plus (residual $\approx +1.7$) and Gemini 2.5 Flash+Think (residual $\approx +3.0$) both fall above it, showing larger-than-expected gaps. Across all eight reasoning-model configurations, the mean residual is -0.04 , compared with $+0.01$ for the 24 standard-model configurations—a difference of 0.05 percentage points. A permutation test (10,000 iterations, two-sided) yields $p = 0.97$, providing no evidence that the residual difference is systematic.

Variance comparison. Average variance (AV) provides a complementary view of trial-to-trial variability. Across the eight reasoning-model configurations, the mean AV is 4.30×10^{-2} , compared with 3.81×10^{-2} for the 24 standard-model configurations—again showing no systematic advantage for reasoning-specialized architectures on this metric.

Interpretation. The four reasoning models do not exhibit systematically smaller accuracy–stability gaps or lower AV once accuracy is accounted for. DeepSeek-R1 shows suggestive evidence of above-expected stability (residual ≈ -4.4 pp), warranting verification in a larger sample. However, with only four reasoning models, the analysis is severely underpowered: the permutation test ($p = 0.97$) should be read as “insufficient evidence from a small sample” rather than “the effect is negligible.” The pattern is model-specific: chain-of-thought reasoning neither reliably improves nor worsens stability within this sample. One important caveat: the Gemini 2.5 Flash vs. Flash+Think comparison is confounded by output token budget (4,096 vs. 24,576 tokens), so the pair should not be interpreted as a clean ablation of thinking mode alone.

A.3 Prompt Sensitivity: Full Analysis

Figure 5 visualizes the prompt effect as a paired scatter plot: each of the 16 models appears as a single point with its MINIMAL-prompt PSR on the x -axis and DETAILED-prompt PSR on the y -axis. Points above the diagonal indicate that detailed prompting improves stability.

Prompt effects are model-dependent: most models show small differences (typically within 1–3 percentage points of PSR), but Claude Sonnet 4.5 shows a reversal (higher PSR under MINIMAL), while Qwen-Turbo shows the opposite (4-point gain under DETAILED). No single template uniformly dominates—consistent with prior findings on prompt sensitivity (Zhuo et al., 2024). Exact McNemar tests on the paired PSR indicators find no model reaching $p < 0.05$; the largest PSR shift is 6 points (o4-mini, whose decoding regime is not user-controllable; see the Limitations section), and among the remaining 15 models the largest shift is 4 points ($p = 0.219$). Across 16 models, 11 favor DETAILED and 5 favor MINIMAL.

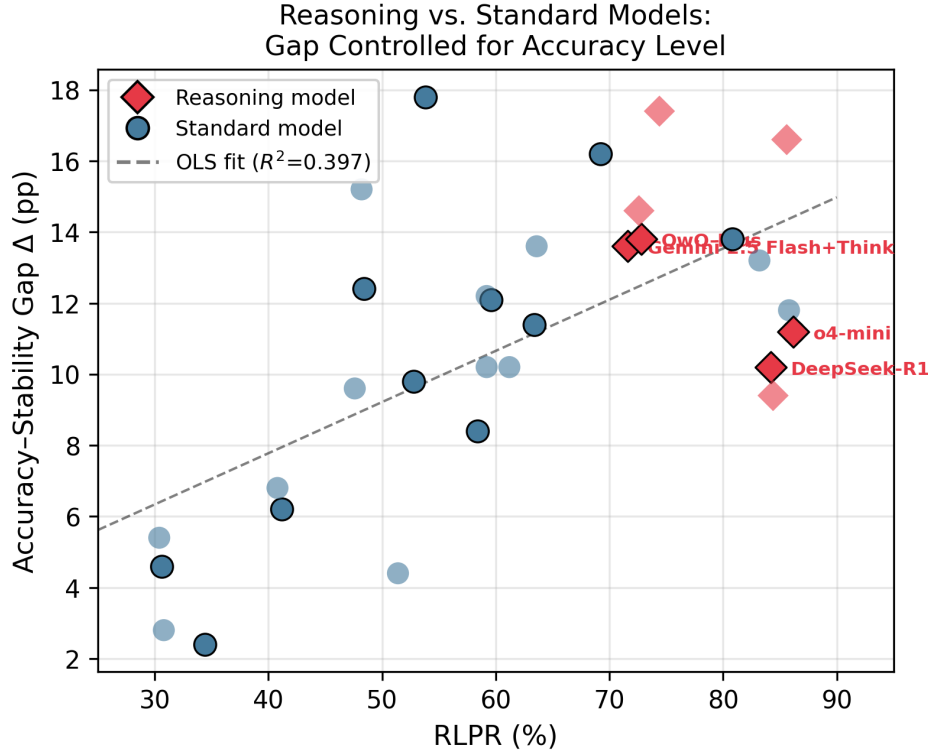


Figure 4: Accuracy–stability gap (Δ) vs. RLPR for all 32 configurations. Reasoning models (diamonds) and standard models (circles) are shown against an OLS fit. Points below the line have smaller gaps than expected at their accuracy level.

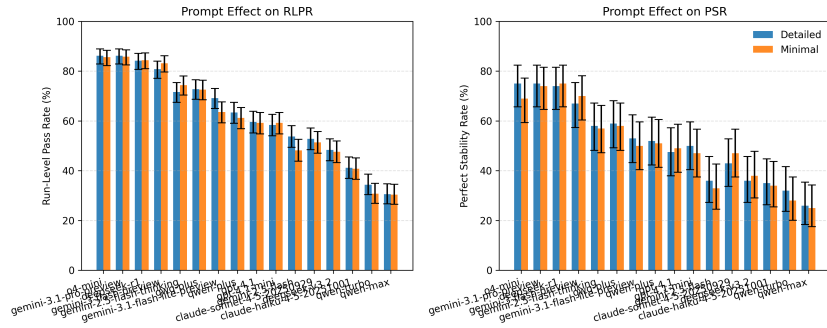


Figure 5: Prompt sensitivity: paired scatter of PSR under DETAILED vs. MINIMAL prompting for all 16 models. Points above the diagonal indicate that detailed prompting improves stability. Points are colored by model family.

480 (mean absolute difference ~ 2.3 pp). This is a deliberately low-contrast comparison—both
 481 templates share identical structure, differing only in instruction density—so the null result
 482 may partly reflect insufficient contrast rather than a true absence of prompt effects. With
 483 $N = 100$, a 4-point shift has less than 20% power, meaning meaningful effects cannot be
 484 ruled out at the current sample size.

B Prompt Templates

B.1 Detailed Prompt

```

<role>
You are an expert Python algorithm engineer solving deterministic
LeetCode-style problems.
</role>

<task>
Produce a correct Python solution that fits the provided template and
is optimized for the problem constraints.
</task>

<instructions>
1. Read the full problem statement, constraints, and template before
   writing code.
2. Produce the best practical algorithm for the stated constraints,
   aiming for the optimal time complexity when possible.
3. Do not return a brute-force, quadratic, exponential, or placeholder
   solution when the constraints require a more efficient algorithm.
4. Use the required method signature from the template exactly.
5. Ensure the algorithm is complete and submission-ready, not a sketch
   or partial attempt.
6. Handle edge cases implied by the statement and constraints.
7. Return one complete Python solution that can be submitted directly.
</instructions>

<format_requirements>
Return plain Python source code only.
Start directly with the code.
Do not include explanations, markdown fences, or surrounding commentary.
The code must be valid, runnable Python 3 with no syntax errors.
The code must fit the provided code template without requiring manual edits.
The final code should be written for Accepted-style performance, not just
sample-case correctness.
Only include comments when they clarify non-obvious logic.
</format_requirements>

<problem>
{problem_description}
</problem>

<code_template>
{code_template}
</code_template>

```

B.2 Minimal Prompt

```

<role>
You are a Python coding assistant solving a deterministic
LeetCode-style problem.
</role>

<task>
Write one correct and efficient Python solution that fits the provided
template.
</task>

<instructions>
1. Infer an algorithm that matches the constraints before writing code.
2. Prefer the optimal or near-optimal solution for the input limits.
3. Do not output brute-force code unless the constraints clearly make

```

```
547     it acceptable.
548 4. Use the exact method signature from the template.
549 5. Return only final submission code.
550 </instructions>
551
552 <format_requirements>
553 Return plain Python source code only.
554 Start directly with the code.
555 Do not include explanations, markdown fences, or extra text.
556 The code must be valid, runnable Python 3 with no syntax errors.
557 The code must fit the provided code template without requiring manual edits.
558 The solution must be efficient enough to avoid obvious Time Limit
559 Exceeded outcomes under the given constraints.
560 </format_requirements>
561
562 <problem>
563 {problem_description}
564 </problem>
565
566 <code_template>
567 {code_template}
568 </code_template>
569
```

570 C Model Snapshots

Table 2: Models evaluated in this study, grouped by provider family. † denotes reasoning-specialized models. Display names are editorial labels derived from API identifier strings; they are not official product version names. Exact API identifiers and access dates are listed in Table 3. Google models marked “(preview)” were accessed via preview-tier API endpoints.

Family	Model	Type
OpenAI	GPT-4.1	flagship
	GPT-4.1-mini	lightweight
	o4-mini	reasoning [†]
Anthropic	Claude Sonnet 4.5	flagship
	Claude Haiku 4.5	lightweight
Google	Gemini 3.1 Pro (preview)	flagship
	Gemini 3 Flash (preview)	efficient
	Gemini 3.1 Flash-Lite (preview)	lightweight
	Gemini 2.5 Flash	efficient
	Gemini 2.5 Flash+Think	reasoning [†]
Qwen	QwQ-Plus	reasoning [†]
	Qwen-Plus	flagship
	Qwen-Max	large
	Qwen-Turbo	lightweight
DeepSeek	DeepSeek-R1	reasoning [†]
	DeepSeek-V3.2	flagship

Table 3: Model snapshots used in the experiments. The API identifier column records the concrete model string used by the evaluation pipeline. Access dates are derived from the archived run dates for the corresponding completed paper runs.

Display Name	Provider	API Identifier	Access Date
GPT-4.1	OpenAI	gpt-4.1	2026-04-01
GPT-4.1-mini	OpenAI	gpt-4.1-mini	2026-03-31
o4-mini	OpenAI	o4-mini	2026-04-06
Claude Sonnet 4.5	Anthropic	claude-sonnet-4-5-20250929	2026-04-03
Claude Haiku 4.5	Anthropic	claude-haiku-4-5-20251001	2026-04-03
Gemini 3.1 Pro (preview)	Google	gemini-3.1-pro-preview	2026-03-28
Gemini 3 Flash (preview)	Google	gemini-3-flash-preview	2026-03-29
Gemini 3.1 Flash-Lite (preview)	Google	gemini-3.1-flash-lite-preview	2026-03-30
Gemini 2.5 Flash	Google	gemini-2.5-flash	2026-04-05
Gemini 2.5 Flash+Think	Google	gemini-2.5-flash (thinkingBudget=8192)	2026-04-05
QwQ-Plus	Qwen	qwq-plus	2026-03-23
Qwen-Plus	Qwen	qwen-plus	2026-03-29
Qwen-Max	Qwen	qwen-max	2026-03-23
Qwen-Turbo	Qwen	qwen-turbo	2026-03-23
DeepSeek-R1	DeepSeek	deepseek-r1	2026-03-24
DeepSeek-V3.2	DeepSeek	deepseek-v3.2	2026-03-25